

HealthTrack Application – Recommendations for Future Improvements and Scaling

Arun Bhaskar Gadde

School of Computing and Information Sciences

University of the Cumberland

2026 Summer - Computer Science Capstone (MSCS-699-B01) - Second Bi-term

Michael Solomon PhD

August 09, 2026

HealthTrack Application – Recommendations for Future Improvements and Scaling

1. High Priority (address before the next major scale-up)

Move from an in-memory cache to a shared cache (Redis)

The current TTLCache lives in one process's memory. The moment HealthTrack runs more than one API worker or process (which is the very next scaling step after this one), each process would have its own separate cache, meaning cache hit rates drop and, worse, different replicas could briefly serve different cached answers for the same patient. Redis (or a managed equivalent) makes the cache shared and consistent across all processes, and is a well-understood, low-risk change since the cache access pattern here (get/set/TTL) maps directly to Redis's basic commands.

Migrate from SQLite to PostgreSQL for production

SQLite was used in this evaluation for zero-setup portability, and it performed well once properly indexed. However, SQLite serializes writes at the file level and is not designed for many concurrent connections from multiple application processes -- both real constraints for a multi-patient, multi-provider monitoring system. PostgreSQL supports proper connection pooling, concurrent writers, and the same indexing strategy validated in this report carries over directly (the query patterns and index design do not need to change, only the connection string and driver).

Run multiple worker processes behind a process manager

The load test showed the optimized dashboard endpoint plateau at ~7.6 requests/second regardless of concurrency (5 vs. 50 simultaneous requests), because a single uvicorn worker process serializes CPU-bound work. Running multiple worker processes (e.g., via gunicorn with uvicorn workers,

sized to available CPU cores) would let independent requests execute in parallel instead of queueing behind one process -- this directly targets the scalability ceiling measured in Section 3 of the Benchmark Results document, and combined with the Redis migration above, is the most direct fix for that specific finding.

2. Medium Priority (plan for, implement as usage grows)

Recommendation	Why
Database connection pooling tuned for concurrent load	Currently uses SQLAlchemy defaults; under multi-worker deployment, pool size should be explicitly sized to (workers x expected concurrent DB calls) to avoid connection exhaustion or excessive idle connections.
Read replicas for reporting/dashboard queries	Aggregate queries like dashboard/summary are read-heavy and read-tolerant of slight staleness; routing them to a replica keeps that load off the primary database serving real-time vitals ingestion.
Background job queue for heavy computations	The predictive-model component of the risk engine (not benchmarked here, since this phase focused on the rule-based scoring path) is more expensive than rule scoring; moving it off the request path into a queue (e.g., Celery/RQ) with the API returning the rule-based result immediately and the ML result shortly after would keep API latency predictable regardless of model cost.
Table partitioning or archiving strategy for vitals_readings	This table grows without bound as more readings are ingested. Partitioning by time (e.g., monthly) or archiving readings older than a defined retention window keeps the

	"hot" table small and query performance stable as total data volume grows well past what was tested here.
CDN for static frontend assets	The minified/gzip-compressed assets in this report were measured for byte size, but were not tested for network delivery; serving them from a CDN edge location reduces latency for geographically distributed users beyond what compression alone achieves.
APM / observability tooling in production	This report's benchmarks were run deliberately, on demand. A production deployment needs continuous latency/error-rate/query-count monitoring (e.g., OpenTelemetry, Sentry, or a hosted APM) so a regression like the N+1 pattern found here is caught automatically, not only when someone happens to benchmark that endpoint again.

3. Lower Priority / Situational

- Async database driver (e.g., asyncpg with PostgreSQL): worth adopting once on Postgres, to let FastAPI's async request handling extend all the way to the database call instead of blocking a thread per request -- most valuable once request volume is high enough that thread-pool exhaustion, not query time, becomes the bottleneck.
- HTTP/2 or HTTP/3 for the API and dashboard reduces connection overhead for clients making many small requests (e.g., a dashboard polling several endpoints) but only matters once the per-request latency work in this report is already solved -- doing this first would have optimized the wrong layer.

- Client-side data caching / stale-while-revalidate on the dashboard frontend: complements, but does not replace, the server-side caching implemented here; most useful once the dashboard is a full SPA rather than the demo page built for this evaluation.
- Rate limiting per API client: not performance optimization per se, but becomes important once the system scales to multiple provider organizations sharing the same infrastructure, to prevent one heavy user from degrading response times for others.

4. What Should NOT Be Done Yet

It is worth explicitly naming what this report does not recommend at the current scale: sharding the database, adopting a microservices split, or introducing a message broker for every write. All three are common "futureproofing" moves that add real operational complexity, and none of them were shown to be necessary by the benchmarks in this report -- the measured bottlenecks (N+1 queries, missing indexes, no caching, no pagination) were all fixable in application code. Introducing infrastructure complexity ahead of a demonstrated need is itself a scaling risk, not a scaling solution.